Check for updates

WILEY

Cladistics

Cladistics (2026) 1–18

doi: 10.1111/cla.70076

Preprocessing missing data and partitions for phylogenetic analysis under dynamic homology, with application to museomics and phylogenomics

Daniel Yudi Miyahara Nakamura[a,b,*], Taran Grant[a] and Ward C. Wheeler[b]

[a]*Instituto de Biociências, Universidade de São Paulo, São Paulo, Brazil;* [b]*Division of Invertebrate Zoology, American Museum of Natural History, New York, New York, USA*

Received 20 May 2026; Revised 3 October 2026; Accepted 7 October 2026

Abstract

In dynamic homology, data must usually be preprocessed to distinguish sequencing artefacts (missing data) from insertion–deletion events, but empirical studies using POY/PhyG have employed manual preprocessing. Here, we compare different procedures for handling missing data for dynamic homology, present Python scripts to automate these procedures, and evaluate their performance using 60 simulated and 4 empirical datasets (including Sanger, phylogenomic and historical DNA sequences). We clarify that coding missing data as gaps or unknown nucleotides/amino acids is theoretically and methodologically inappropriate. In contrast, isolating missing data via partitioning or coding it as any observed state or no state ("?", the maximally polymorphic state; N + gap for nucleotides, X + gap for amino acids) is defensible but involves trade-offs. Partitioning can decrease time complexity but weakens homology testing by isolating potentially homologous nucleotides in different partitions. Conversely, question marks enable unconstrained homology testing but increase time complexity. To balance these trade-offs, we propose heuristic strategies to select optimal partitioning. Our methods reduced time complexity in large simulated datasets and improved tree costs in all empirical datasets, highlighting the importance of properly identifying missing data in dynamic homology analysis.

© 2026 The Author(s). *Cladistics* published by John Wiley & Sons Ltd on behalf of Willi Hennig Society.

Background

Two general approaches have emerged to align sequence data and identify homologous characters and character states. The static homology approach identifies homologous characters *a priori* by performing multiple sequence alignment (MSA) using a similarity function decoupled from tree optimality (i.e. similarity alignment), optimizing that alignment on trees, and searching for the optimal tree to explain that single alignment. In contrast, the dynamic homology approach optimizes unaligned sequences directly onto each tree (i.e. tree alignment) during the search for the optimal tree to explain the observed sequences (Wheeler, 1996, 2001, 2006; Wheeler et al., 2006). Homologous characters and character states are then identified *a posteriori* by generating either the list of transformations at each node or the implied alignment for the optimal tree (Wheeler, 2003c; Washburn and Wheeler, 2020). The problem of finding the optimal tree for a given set of unaligned sequences, transformation costs and an optimality criterion is referred to in combinatorial optimization as the Generalized Tree Alignment Problem (GTAP; Sankoff, 1975; Sankoff et al., 1976; Hein, 1989, 1990; Wheeler, 1996; Varón and Wheeler, 2012, 2013). TAP (Wang and Jiang, 1994) and GTAP are NP-hard (Foulds and Graham, 1982; Varón and Wheeler, 2013), and exact solutions (e.g. Sankoff, 1975; Sankoff and Rousseau, 1975) are infeasible for most empirical datasets (reviewed in

*Corresponding author.
E-mail address: dani_ymn@alumni.usp.br

© 2026 The Author(s). *Cladistics* published by John Wiley & Sons Ltd on behalf of Willi Hennig Society.
This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

Wheeler, 2005, 2012). Consequently, several heuristic algorithms have been proposed to solve GTAP, including direct optimization (DO; Wheeler, 1996; Varón and Wheeler, 2012, 2013), iterative-pass optimization (Wheeler, 2003a), and search-based optimization (Wheeler, 2003b).

In empirical studies, data can be absent due to either evolutionary insertion–deletion events (indels) or technical limitations. Since all sequence-length variation is treated as real and explained by indels in tree alignment, it is imperative that artifactual missing data be identified and correctly coded prior to tree alignment to avoid grouping terminals on the basis of sequencing, assembly and concatenation artefacts. For example, in DNA sequence analysis, whole loci can be missing due to technical problems that prevent their detection, such as ineffective primers or probes, copy number variation or other factors, and portions of loci can be missing due to the elimination of low-quality bases at flanking positions, use of different sets of primers with no overlap among amplicons, or inadequate coverage in assemblies of high-throughput DNA sequences (Wheeler, 2012; Hosner et al., 2016). As such, although input sequences are unaligned under tree alignment, most empirical analyses employ a preliminary similarity-based MSA to identify data that are absent due to technical limitations (e.g. Giribet, 2001, 2002; Frost et al., 2006; Grant et al., 2006; Wheeler et al., 2014, 2017; Peloso et al., 2015).

When each locus is input as a separate file, missing loci are treated as zero-length partitions for the terminals that lack them; however, loci that vary in sequence length due to missing data must be preprocessed through a combination of three approaches, each with its own trade-offs (e.g. Frost et al., 2006; Grant et al., 2006; Wheeler et al., 2014, 2017). First, complete sequences can be trimmed to match the length of incomplete sequences, the obvious problem being the exclusion of potentially important evidence. Second, incomplete sequences can be extended to match the length of complete sequences. In early versions of POY, this was accomplished by inserting nucleotides of unknown identity (IUPAC N); however, although this approach does not constrain the identity of unsequenced nucleotides, it assumes those sequences are actually the same length, which is not necessarily correct. Consequently, POY 5 (Wheeler et al., 2015) and PhyG (Wheeler et al., 2024) allow sequences to be extended by inserting question marks (?) to represent any observed state or no state (i.e. the maximally polymorphic state; N+ or gap for nucleotides, X or gap for amino acids; Wheeler, 2012). Third, blocks of missing data can be isolated by inserting hash signs (#) to break sequences into smaller fragments, with adjacent hash signs indicating that the intervening data are missing (e.g. Giribet et al., 2000; Edgecombe et al., 2002). This approach is especially efficient when large fragments are missing due to different primer sets being used in different studies and has the additional advantages of accelerating analyses and reducing memory requirements by reducing alignment space (Wheeler et al., 2006: 108–110), meaning that it can be advantageous to insert breaks even if sequences are complete for all terminals. Nevertheless, this approach constrains homology assessment by preventing data from adjacent fragments from being aligned, which can result in suboptimal solutions.

Previous studies have pre-processed sequences manually without explicitly assessing the impact of choices on homology, tree cost or topology. As a result, the best approach for dealing with missing data in dynamic homology needs to be clarified. Moreover, manual preprocessing increases subjectivity, decreases replicability and can be extremely time-consuming and error-prone. Ideally, data preprocessing should be automated to facilitate input file formatting in POY (Wheeler et al., 2015) and PhyG (Wheeler et al., 2024). This is especially relevant in phylogenomics with hundreds to thousands of gene alignments and studies of degraded historical (hDNA) and ancient DNA (aDNA), which present a high proportion of variably distributed missing data.

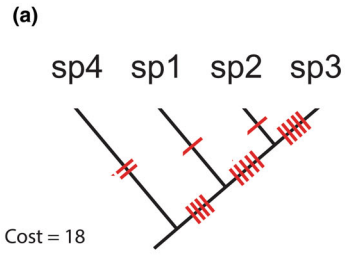
Here, we (1) compare different procedures to deal with missing data prior to tree alignment, (2) present a script to automate input-sequence preprocessing for POY/PhyG, and (3) test our script using simulated and empirical data and the DO algorithm.

Preliminaries

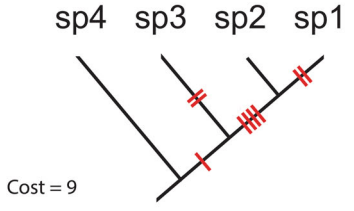
The hypothetical examples illustrate the consequences and trade-offs of analysing sequences with different strategies (Figs 1–3). All dashes present in preliminary static alignments (input data) are automatically deleted before running DO (unless specified as `prealigned`; see explanation in Wheeler et al., 2017). We ran DO with equally weighted parsimony using `build()`, `swap()` and `refine()` and reported output tree topologies and implied alignments using PhyG v1.3 (Wheeler et al., 2024).

Consider the five procedures in input data illustrated in Fig. 1: no preprocessing (Fig. 1a), trimming complete sequences (Fig. 1b), extending incomplete sequences using IUPAC N (Fig. 1c), creating zero-length partitions with hash signs (#) to isolate blocks of missing data (Fig. 1d), and extending incomplete sequences using question marks (Fig. 1e). In Fig. 1a, DO results in a tree cost of 18 because 6 indel events are inferred to explain the difference in sequence length (i.e. although artifactually missing, PhyG explains the absent flanking nucleotides as indels due

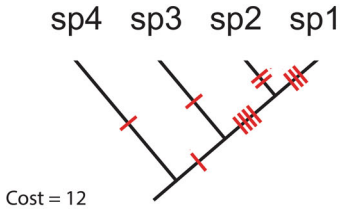
10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



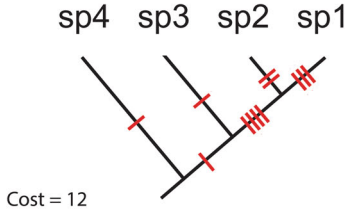
Input data	Implied alignment
>sp1 GGGGACGTCTTTAG >sp2 GGGGAGGAA >sp3 AAAAAAAA >sp4 AAAAAGGTTTCTAG	>sp1 GGGGACGTCTT-TAG >sp2 GGGGAGG------AA >sp3 -AAAAAA------AA >sp4 AAAAAGGT-TTCTAG



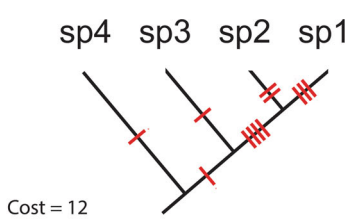
Input data	Implied alignment
>sp1 GGGGACGT >sp2 GGGGAGGA >sp3 AAAAAAAA >sp4 AAAAAGGT	>sp1 GGGGACGT >sp2 GGGGAGGA >sp3 AAAAAAAA >sp4 AAAAAGGT



Input data	Implied alignment
>sp1 GGGGACGTCTTTAG >sp2 GGGGAGGAANNNNN >sp3 AAAAAAAANNNNNN >sp4 AAAAAGGTTTCTAG	>sp1 GGGGACGTCTTTAG >sp2 GGGGAGGAANNNNN >sp3 AAAAAAAANNNNNN >sp4 AAAAAGGTTTCTAG



Input data	Implied alignment
>sp1 GGGGACGT#C#TTTAG >sp2 GGGGAGGA#A#----- >sp3 AAAAAAAA#-#----- >sp4 AAAAAGGT#T#TCTAG	>sp1 GGGGACGTCTTTAG >sp2 GGGGAGGAA????? >sp3 AAAAAAAA?????? >sp4 AAAAAGGTTTCTAG



Input data	Implied alignment
>sp1 GGGGACGTCTTTAG >sp2 GGGGAGGAA????? >sp3 AAAAAAAA?????? >sp4 AAAAAGGTTTCTAG	>sp1 GGGGACGTCTTTAG >sp2 GGGGAGGAA????? >sp3 AAAAAAAA?????? >sp4 AAAAAGGTTTCTAG

Fig. 1. Theoretical example comparing (a) no preprocessing, (b) trimming complete sequences, (c) extending incomplete sequences with IUPAC, (d) creating zero-length partitions, (e) coding with question marks. Dashes present in the input data are deleted before running DO. The red lines indicate transformations, depicting one of the possible optimizations.

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

to the lack of preprocessing), favouring the grouping of *sp2* and *sp3*. In Fig. 1b–e, the grouping of *sp1* and *sp2* is favoured, with a decrease in cost to 12 in Fig. 1c–e. The cost of 9 steps in Fig. 1b is not comparable to the others because data were deleted, resulting in a loss of information.

Although topologies and costs are the same in Fig. 1c–e, missing data are not treated as such in Fig. 1c, in which incomplete sequences are extended with IUPAC Ns. The IUPAC N assumes that a nucleotide of unknown identity is present without considering the possibility of sequence length variation (gap, −), because N = {A,C,G,T}. The disadvantage of coding missing data inappropriately with IUPAC Ns is highlighted in Fig. 2, in which topologies changed, and tree costs increased. If missing data are coded with question marks or isolated in zero-length partitions, missing values are interpreted as maximally polymorphic states, ? = {A,C,G,T,–} (Wheeler, 2012). Although question marks (?) are interpreted in many programs as any nucleotide or a gap, the IUPAC system does not provide a standardized symbol for missing data. Furthermore, the absence of an official IUPAC symbol for missing values has led to their frequent misrepresentation as "N" in sequences deposited in GenBank (e.g. positions in hDNA sequences with little or no coverage support). The IUPAC symbol "N" indicates any nucleotide but excludes the possibility of a gap. Therefore, we recommend that IUPAC establish a standardized symbol for missing data in their alphabet. Thus, fewer assumptions are required by not assuming the existence of nucleotides. Minimizing these *ad hoc* hypotheses is particularly relevant for hDNA sequences.

Both question marks and partitions are appropriate to code missing data, but partitioning constrains tree alignment because homology can only be tested within the same partition, not across partitions (Wheeler et al., 2006). For instance, Fig. 3 shows missing data coded with question marks (Fig. 3a) and zero-length partitioning (Fig. 3b). Even though the string TCAGTCGATGCATGCTAGCT is present in both *sp2* and *sp3*, homology cannot be tested in Fig. 3b because portions of this fragment are isolated in different partitions, leading to a higher tree cost and different topology than when homology testing is unconstrained (Fig. 3a). As such, more assumptions are requested by isolating missing data with hash signs than coding them with question marks.

Method

We present a collection of Python scripts to facilitate the preprocessing of DNA sequences for tree alignment (Fig. 4). Although each function can be executed separately, the script `prepDyn.py` ("preprocessing dynamic characters") integrates all functions into a straight-forward pipeline of four steps: (1) data collection, (2) trimming, (3) identification of missing data and (4) successive partitioning. Modifications in each step can be reported in a log file (`log True`). Additional scripts are also available to align one or a few sequences to a profile alignment (i.e. a previously preprocessed file in POY/PhyG format; `addSeq.py`), align sequences containing non-IUPAC symbols (viz., question marks and hash signs;

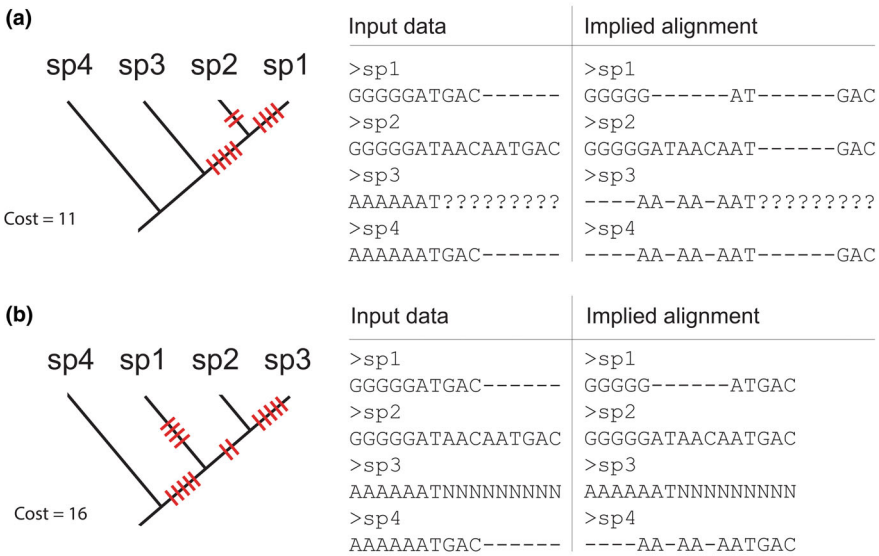


Fig. 2. Theoretical example showing that coding missing data with (a) question marks instead of (b) IUPAC N can reduce tree costs in direct optimization. Dashes present in input data are deleted before running DO. The red lines (steps on tree) depict one of the possible optimizations. In the implied alignment, columns containing only "–" and "?" do not contribute to tree costs. The presence of question marks implies the possibility of a nucleotide, so POY/PhyG must account for those positions by adding additional gaps in other sequences (even though those gaps do not contribute to the cost).

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



Fig. 3. Theoretical example showing that (a) coding missing data with question marks instead of (b) isolating blocks of missing data with hash signs can reduce tree costs in direct optimization. Dashes present in input data are deleted before running DO. The red lines (steps on tree) depict one of the possible optimizations. See explanation of columns containing only gaps and question marks in Fig. 2.

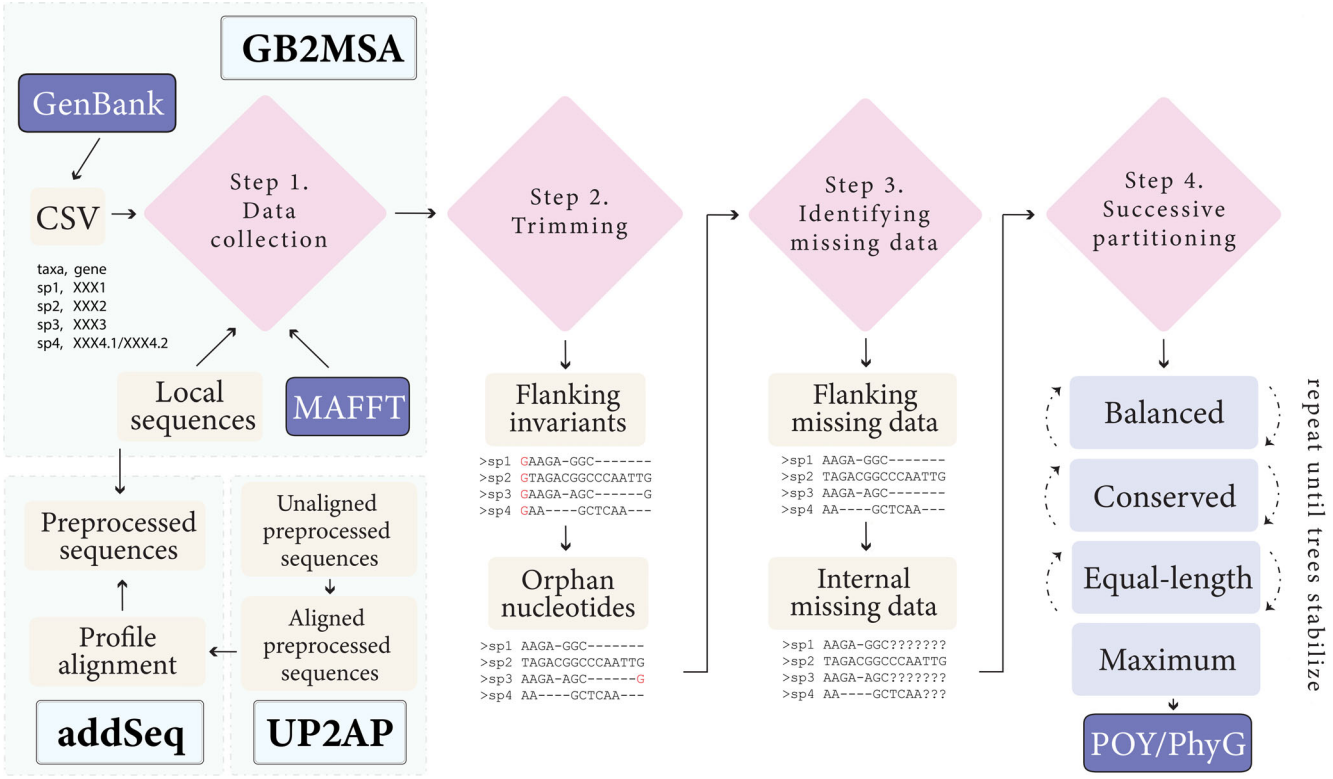


Fig. 4. Workflow of *prepDyn*.

`UP2AP.py`), or fetch data from GenBank (`GB2MSA.py`). The scripts were tested on macOS, Linux and Windows Subsystem for Linux (WSL). Instructions for installation (for manual, Conda, or Docker setups) and tutorials are available at https://github.com/dnakamuraz/PrepDyn.

Step 1: Data collection

Using the parameter `CSV_input`, fetching sequences from GenBank is possible by inputting a CSV dataframe containing accession numbers. For genes amplified with more than one set of primers (e.g. mitochondrial 16S rRNA), two or more GenBank accession numbers can be specified in the same cell and delimited with a pipe (e.g. MT893619|MT895696). If these amplicons are non-overlapping, the index of intersequence dashes is stored for *Step 3*. Otherwise, if amplicons overlap, *prepDyn* identifies the overlapping nucleotides via pairwise alignment (permitted mismatch percentage user-specified with `multi_amplicon_mismatch_rate`) and either deletes the overlapping region from one of the amplicons (`multi_amplicon_action trim`) or calls the consensus of the overlapping sequences using IUPAC ambiguities (`multi_amplicon_action consensus`).

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

If the user only possesses local sequence files, the parameter `GB_input None` skips data collection from GenBank and `input_file` parses local files, which can be either a single MSA (Fasta, Nexus, Phylip, or other formats accepted by Biopython; Cock et al., 2009) or a directory containing multiple MSAs.

If the user possesses local files and GenBank accession numbers, both sources of data can be specified in `CSV_input` (see https://github.com/dnakamuraz/prepDyn/wiki/Tutorial-1). This is convenient for combining sequences from GenBank, sequences available in other databases (e.g. phylogenomic data containing hundreds or thousands of gene alignments, usually made available by systematists in Dryad and Zenodo instead of GenBank), and newly sequenced data. We recommend annotating mitogenomes and parsing each gene in individual columns in the CSV file (instead of specifying accession numbers of whole mitogenomes) to avoid problems with orthology assignment due to gene order rearrangement. Finally, a preliminary MSA is generated using either MAFFT (`aligner mafft`; default; Katoh and Toh, 2008) or Clustal W (`aligner clustalw`; Larkin et al., 2007). *Step 1* can also be performed as a standalone function (skipping *Steps 2–4*) using the script `GB2MSA.py`.

Step 2: Trimming

Deletion of internal (= non-flanking) nucleotides is not appropriate in dynamic homology because they would be misinterpreted as indels (see comments in Peloso et al., 2015). In contrast, trimming trivial nucleotides on flanking positions (i.e. start and end positions) can facilitate next steps and reduce runtime. Here, we provide two options for trimming. First, the script can automatically trim flanking invariant columns (`del_inv True`) that are not expected to affect tree choice. Second, low-quality data (e.g. errors from sequencing) concentrated at the ends of sequences can generate alignment artefacts. Specifically, these artefacts can emerge as "orphan nucleotides" (i.e. single or small blocks of nucleotides isolated by many dashes from the core block of nucleotides due to artefacts) that we recommend trimming. Orphan nucleotides are defined as blocks of contiguous flanking nucleotides of length less than x and isolated from the adjacent block of nucleotides by contiguous dashes of length longer than x. Given that the threshold x is data-dependent, it can be specified through a quick visual inspection of the alignment (`orphan_threshold`). We recommend this option for the mitochondrial loci 12S and 16S rRNA obtained through Sanger sequencing, in which orphan nucleotides are frequent. In our experience, a threshold x of 25–45 is appropriate in most cases. If the chromatograms of Sanger sequences or read mappings of high-throughput DNA sequences are available, orphan nucleotides can be validated by confirming low base quality or low coverage depth. Alternatively, if orphan nucleotides are determined to be due to MSA errors rather than sequencing errors, they can be realigned adjacent to the next block (`orphan_action push`) instead of trimmed (`orphan_action trim`).

Step 3: Identification of missing data

All terminal dashes (i.e. contiguous strings of dashes at the start and end positions of sequences) are missing data, so *prepDyn* automatically replaces them with question marks. In contrast, internal dashes can correspond to either indels or missing data. Specifically, when concatenating non-overlapping fragments, internal dashes between these fragments are missing data, whereas other internal dashes within (not between) these fragments result from putative insertion–deletion events, which will be validated by DO. Internal missing data can also result from low coverage depth in high-throughput sequences. If multiple sequences from the same locus and sample have no overlap between them, intersequence dashes are automatically converted to question marks (i.e. treated as internal missing data). In hDNA, regions in the consensus sequence that are poorly supported by reads are usually reported as IUPAC N. Although a question mark can be specified instead of IUPAC Ns in consensus calling of some programs (e.g. Geneious; Kearse et al., 2012), GenBank only uses IUPAC symbols, so they are reported as IUPAC N. To minimize assumptions, all IUPAC Ns present in hDNA can be replaced with question marks in *prepDyn* by specifying the hDNA terminals in the parameter `n2question`.

Manual methods to handle internal missing data are also available in *prepDyn*. For instance, the user can force the replacement of dashes with question marks in specific positions (e.g. `internal_method manual, internal_column_ranges 150-180, 250-280, internal_leaves all`) or replace all blocks of internal dashes greater than y (e.g. `internal_method semi, internal_threshold 30`). Background knowledge about which primers were used or how sequences were concatenated is necessary if the user skips *Step 1*; otherwise, manual methods are unnecessary and internal missing data are automatically identified by *prepDyn*.

Step 4: Partitioning

In *Preliminaries* (Figs 1–3), we demonstrated that identifying missing data both by isolating blocks of missing data via partitioning and inserting question marks is appropriate, but that partitioning constrains homology testing. However, breaking sequences into smaller fragments decreases time complexity (the higher the number of partitions, the lower the time complexity), which can enable more thorough tree searching. Giribet (2001) provided empirical examples in which partitioning helps reduce tree cost by accelerating searches. As such, there is a trade-off between these approaches (Wheeler et al., 2006).

Another advantage of breaking sequences into smaller partitions is that memory usage is reduced. Indeed, if large datasets are not partitioned, long sequences (e.g. >100 000 nucleotides) are not tractable with POY/PhyG due to memory constraints, and some compilation versions of POY impose a restriction of 100 000 nucleotides per partition. Partitioning long sequences by inserting hash signs enables them to be analysed. For example, a maximum partition size (`pms`) of 10 000 nucleotides is set by `prepDyn -pms 10000`.

Optimality scores are not comparable for datasets preprocessed with different methods of trimming or missing data identification (*Steps* 1–3). In contrast, given the same set of sequences pre-processed with the same parameters in *Steps* 1–3, the optimality scores that result from different partitioning schemes (*Step* 4) are numerically competitive in dynamic homology. Thus, different partitioning schemes can be tested to boost heuristics. We propose four heuristic methods to automate partitioning and guide where to place hash signs. Three of them are based on successive partitioning, in which the number of partitions (`pr`) increases in each round until heuristically optimal trees become stable, thereby guiding how many partitions to create. The differences in optimal trees between successive rounds can be operationalized as tree cost differences, distances between strict consensus topologies, or a combination of both. Furthermore, stability can be operationalized as trees stable across two successive rounds or multiple successive rounds. The higher the number of successive rounds in which trees are stable, the more severe the test. Ideally, the initial value of `pr` should be zero (no partitioning), but in practice the initial `pr` is usually combined with `pms` considering dataset size and availability of time and computational resources. The final value of `pr` should be selected by the lack of improvement in tree costs (and/or no topological differences). Likewise, the effect of different methods of partitioning can be tested by analysing each partition scheme in POY/PhyG, with the method that results in the lowest-cost tree being optimal.

The four heuristic methods of partitioning are described below (Fig. 5):

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

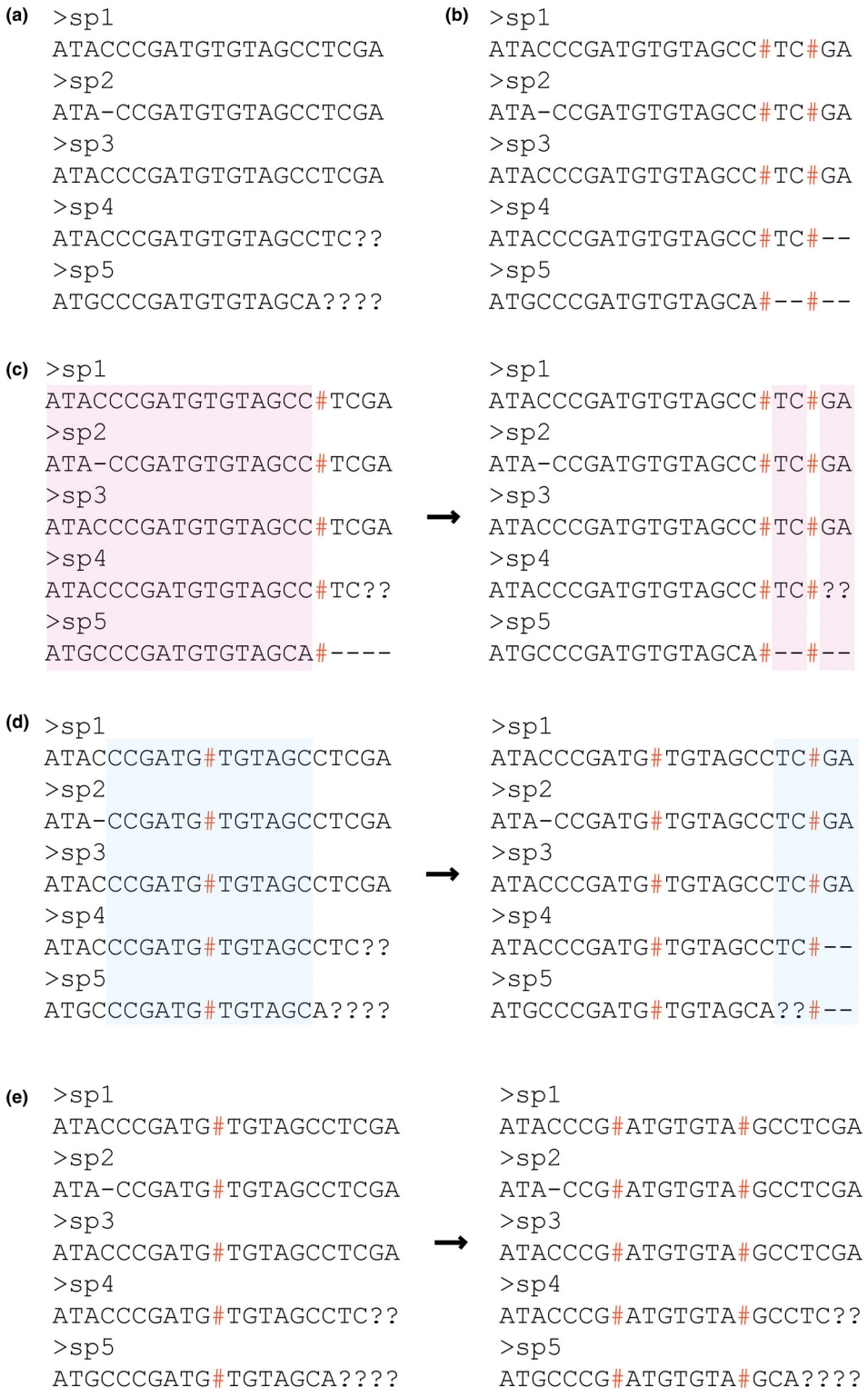


Fig. 5. Comparison of partitioning strategies using a simple example. (a) No partitioning. (b) Maximum partitioning. (c) Balanced partitioning (the purple rectangle depicts the n-longest block after maximum partitioning, that is, the longest block in the first round and the second-longest block in the second round; their sizes serve as thresholds for merging adjacent blocks whose combined length is smaller than the corresponding threshold). (d) Conservative partitioning (the blue rectangle depicts the n-longest invariant block, that is, the longest block of invariants in the first round and the second longest in the second round, which are partitioned at the midpoints). (e) Equal-length partitioning. Two rounds of successive partitioning are depicted in c, d and e (first round on left, second round on right).

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

- `prepDyn -pm max` adds a hash sign column flanking all blocks of missing data (Algorithm 1). This is expected to be the fastest method, although it also imposes the greatest constraint on nucleotide homology.
- `prepDyn -pm balanced` is based on successive rounds of partitioning (`-pr`; Algorithm 2). Initially, *prepDyn* inserts hash signs around all blocks of missing data (as in `-pm max`) and sorts the length of these blocks. Then, the length of the n-largest block is stored (e.g. the length of the largest block is stored if `pr 1`, the length of the second-largest block is stored if `pr 2`, and so on). Next, each block is inspected iteratively: if the sum of the current and adjacent block is larger than the length of the n-largest block, the hash sign is retained; otherwise, the hash sign is deleted and blocks are merged. This procedure continues until all blocks have been evaluated.
- `prepDyn -pm conservative` inserts hash signs in highly conserved regions based on successive partitioning rounds (`pr`; Algorithm 3). Initially, *prepDyn* sorts all blocks of contiguous invariant columns and saves their start and end position indexes. If `pr = 1`, the midpoint of the longest block of contiguous invariant sites is selected for partitioning. When multiple blocks share the maximum length, each is partitioned accordingly. If `pr = 2`, the longest and second-longest blocks of invariants are partitioned and so on. Alternatively, by specifying `partitioning_conservative flank`, hash signs can be inserted around invariant blocks instead of their midpoints (`conservative_method midpoint`).
- `prepDyn -pm equal` generates equal-length partitions (Algorithm 4). The number of hash signs inserted is specified by `pr` (e.g. `pr 1` inserts one hash sign to divide the sequence into two equal-length partitions).

All partitioning methods and multiple rounds can be executed with a single command (e.g. `prepDyn -pm all -pr 0-3` partitions DNA sequences using `-pm balanced`, `-pm conservative` and `-pm equal` for `-pr 0`, `-pr 1`, `-pr 2` and `-pr 3`, and `-pm max`).

Algorithm 1: Maximum partitioning around blocks of missing data.

Input: Preprocessed DNA sequences, *M*

Output: *M′* including '#' flanking contiguous strings of '?'

L ← length (*M*) // Length of sequences

pos [1 .. *L*] ← F // Boolean vector indicating where '#' will be inserted

for *sequence* **in** *M* **do** // Set '#' insertion positions in *M*

  i ← 1

  while *i* ≤ *L* **do**

    if *sequence*[*i*] = '?' **then**

      start ← *i* // Position where missing block opens

      while *i* ≤ *L* **and** *sequence*[*i*] = '?' **do**

        i ← *i* + 1

      end ← *i* // Position after the missing block

      if *start* > 1 **then** *pos*[*start*] ← T

      if *end* ≤ *L* **then** *pos*[*end*] ← T

    else *i* ← *i* + 1

  for id, *sequence* **in** *M* **do**

    new ← empty list

    for *i* = 1 .. *L* **do**

      if *pos*[*i*] = T, **then** append ('#', *new*) // Insert '#' in *M* at positions indicated in *pos*

      append (*sequence*[*i*], *new*) // If false in *pos*, only append the bases

    M′ [id] ← join (*new*)

return *M′*

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Algorithm 2: Balanced partitioning via merging threshold.

Input: Preprocessed DNA sequences, M

Input: Partitioning round, $pr \in \mathbb{N}$

Output: M' containing '#' flanking every block of '?' unless the combined length of adjacent blocks is less than the length of the pr-th largest block(s)

$L \leftarrow$ length (M) // Length of sequences

if $pr = 0$ **then**

  $M' \leftarrow M$

else

  $M' \leftarrow$ max_partitioning (M) // Run Algorithm 1

  $pos \leftarrow \{ i : \forall j, M'[j,i] = \text{'\#'} \}$ // Find the '#' positions in M'

  $B \leftarrow \{ (pos_k, pos_{(k+1)}) : k = 0,\ldots,|pos|-2 \}$ // Find start and end positions of partitions

  $\ell(b) \leftarrow$length(b) // Compute the length of each partition b in B

  $(b_1, \ldots, b_m) \leftarrow$ sort (B, ℓ) // Sort partitions

  $N \leftarrow \ell(b_{pr})$ // Define the merging threshold as the pr-th largest block length

  define merge (*direction*) // Define the function to merge partitions

    changed $\leftarrow$ T

    while changed = T **do**

      changed $\leftarrow$ F

      recompute (pos, B) // Account for shifts

      if *direction* = $\rightarrow$ **then** k = 1…$|B|-1$ **else** k = $|B|-1$…1 // Define block indices

      for k $\in$ k_range **do** // Iterate each pair of adjacent partitions

        $B_k = (s_1, e_1)$ // Define the limits of the partition k in B

        $B_{(k+1)} = (s_2, e_2)$ // Define the limits of the next partition $k+1$ in B

        if $\ell(B_k) + \ell(B_{(k+1)}) < N$ **then**

          delete '#' at e_1 // merge adjacent blocks

          changed $\leftarrow$ T

          break

  merge ($\rightarrow$); merge ($\leftarrow$)

return M'

Simulations

To clarify the time complexity and behaviour of partitioning on preprocessing and downstream DO analyses, we analysed 60 simulated datasets. Initially, we simulated phylogenetic trees with variable numbers of leaves (10, 20, 40 and 80) using *ngesh* (Tresoldi, 2021) and DNA sequences with variable numbers of

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Algorithm 3: Conserved partitioning.

Input: Preprocessed DNA sequences, M
Input: Partitioning round, $pr \in \mathbb{N}$
Output: M' containing '#' at the midpoints of the pr largest invariant block(s)
$L \leftarrow$ length (M); $r \leftarrow$ nrow (M) / Length and number of sequences
$M' \leftarrow M$
if $pr = 0$ **then**
  return M'
else
  // Classify positions as variants and invariants
  for i **in** $1 .. L$ **do**
    if $|\{ sequence_{k[i]} \mid k = 1..r \} \setminus \{'?'\}| = 1$ **then** $type[i] \leftarrow$ invariant
    else $type[i] \leftarrow$ variant
  // Find contiguous invariants blocks
  $B \leftarrow \emptyset$; $i \leftarrow 1$ // Empty B intervals between '#'
  while $i \leq L$ **do**
    if $type[i]$ = invariant **then**
      $s \leftarrow i$ // Block start
      while $i \leq L$ **and** $type[i]$ = invariant **do**
        $i \leftarrow i + 1$
      $e \leftarrow i - 1$ // Block end
      $B \leftarrow B \cup \{(s,e)\}$ // Store invariant block
    else $i \leftarrow i + 1$
  // Select the pr largest invariant block(s)
  $B_sorted \leftarrow$ sort (B, descending = T)
  $H \leftarrow \emptyset$ // Positions where '#' will be inserted
  for s,e **in** B_sorted $[1 .. pr]$ **do**
    $m \leftarrow \lfloor (s + e) / 2 \rfloor$ // Midpoint of block
    $H \leftarrow H \cup \{m\}$ // Final '#' positions
  // Partition M' at selected midpoints
  sort (H)
  for $sequence_k$ **in** M' **do**
    $shift \leftarrow 0$
    for h **in** H **do**
      insert ('#', position = $h + shift$ **in** $sequence_k$)
      $shift \leftarrow shift + 1$
return M'

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Algorithm 4: Equal-length partitioning.

Input: Preprocessed DNA sequences, M

Input: Partitioning size, $ps \in \mathbb{N}$, **or** partitioning round, $pr \in \mathbb{N}$

Output: M' partitioned into equal-length blocks of size ps **or** divided into $pr + 1$ blocks of equal size

$L \leftarrow$ length (M) // length of sequences

$pos \leftarrow \emptyset$ // empty set of insertion positions

if $ps = 0$ **or** $pr = 0$ **then**

  $M' \leftarrow M$

else

  // Set '#' insertion positions in M

  if $ps \neq 0$ **then**

    $pos \leftarrow \{ k \cdot ps: 1 \leq k < L / ps \}$ // Set '#' insertion positions in multiples of ps

  elif $pr \neq 0$ **then**

    $interval \leftarrow \lfloor L / (pr + 1) \rfloor$ // Divide the sequences into $pr + 1$ partitions of equal size

    $pos \leftarrow \{ (i+1) \cdot interval : i = 0,\ldots,pr-1 \}$ // Set '#' positions at internal boundaries

  $pos \leftarrow$ sort (pos)

  // Insert '#' in M at positions indicated in pos, accounting for shifts in each insertion

  for *id, sequence* **in** M **do**

    $seq \leftarrow$ list ($sequence$)

    $shift \leftarrow 0$

    for p **in** pos **do**

      insert ('#', seq, position = $p + shift$)

      $shift \leftarrow shift + 1$

    M' [id] $\leftarrow$ join (seq)

return M'

nucleotides (100, 1000 and 10 000) using the command `iqtree3 --alisim alignment_indel -m "JC" -t tree_{n}.nwk --length {length} --indel 0,0.1 --out-format fasta` (Ly-Trong et al., 2022). To control the number and size of partitions, we partitioned the data using equal-length partitions (`-pr` 0, 1, 2, 4, 8). Finally, we conducted DO analyses of unaligned sequences using the commands `build` and `swap` with default parameters (and no time constraint) in PhyG v. 1.3 (Wheeler et al., 2024). We $\log_2$-transformed values and ran regression analyses considering input size (number of leaves, total sequence length and number of partitions) and runtime in CPU seconds (CPU sec.) and tree cost as predictor and response variables, respectively. We performed all analyses on the "hel" cluster housed at the American Museum of Natural History with 48 cores (dual CPU) AMD EPYC 7513 2.6Ghz servers with 4TB RAM running CentOS 7.

For preprocessing, the runtime of *prepDyn* was 0.01–2.07 CPU sec. (1.83–2.07 CPU sec. in the largest simulated dataset fixing 80 leaves and ca. 10 000 nucleotides but varying the number of partitions). Considering n as

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

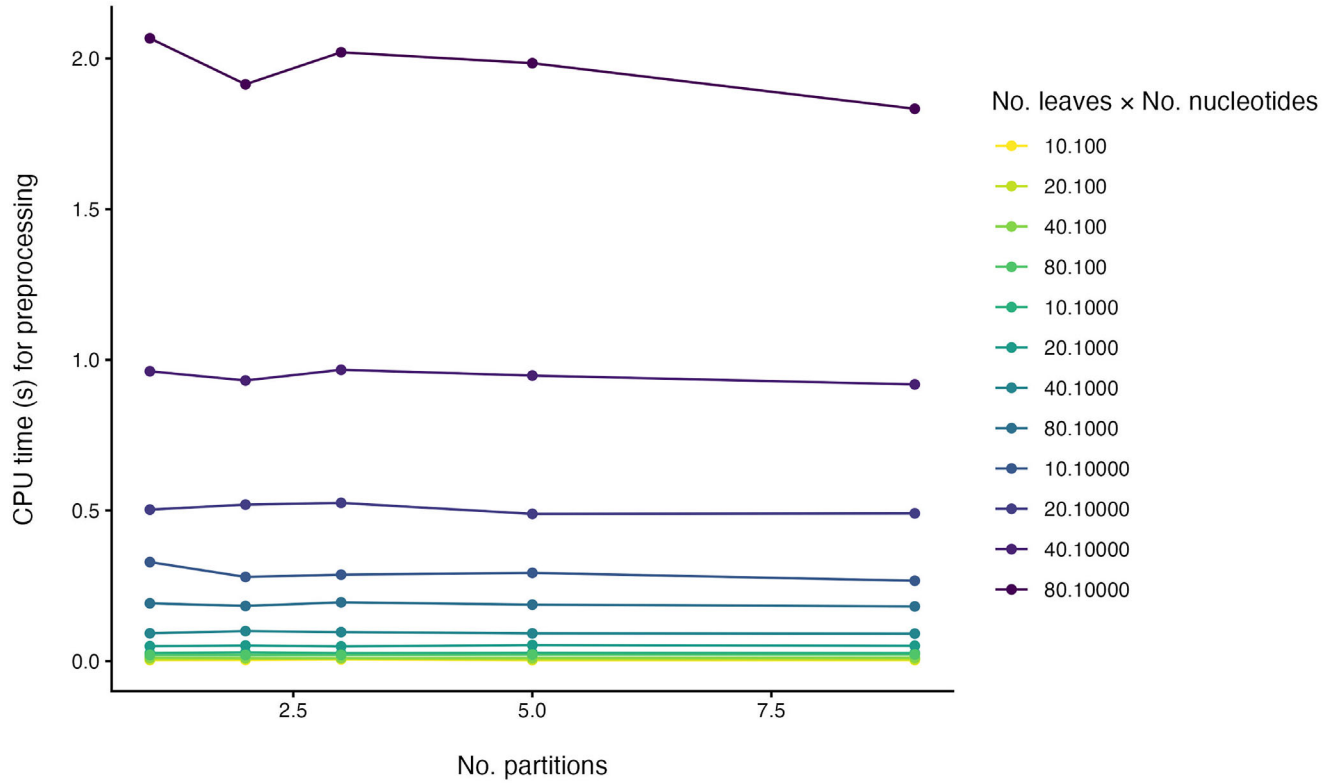


Fig. 6. Relationship between preprocessing (*prepDyn*) CPU runtime and number of partitions, considering number of leaves (10, 20, 40, and 80) and total sequence length (100, 1000, and 10 000) using simulated data. See also Appendix S1: Fig. S1.

number of leaves, m as total sequence length, and k as number of partitions, the regression model (adjusted $R^2 = 0.99$) revealed that the runtime to automatically preprocess data using *prepDyn* grows linearly with n (estimate = 0.88; $P < 0.001$) and m (estimate = 0.95; $P < 0.001$), whereas k is not a significant predictor ($P = 0.37$; Fig. 6; Appendix S1: Fig. S1).

For downstream phylogenetic analyses using DO, the runtime was 188–42 494 920 CPU sec. (29569356–42 494 920 CPU sec. in the largest dataset). The time complexity of phylogenetic analyses is typically O ($n^{3–4}$ m^{x}), where n is the number of leaves and m is the number of nucleotides ($x = 1$ if homology is static, $x > 1$ if homology is dynamic; Wheeler, 2025). Our simulations corroborate Wheeler (2025), with $n^{3.39}$ and $m^{1.21}$ (Appendix S1: Tables S1 and S2). When we take into account the k number of hash signs dividing sequences into equal-length partitions, our regression model considering the largest dataset revealed that k slightly decreases runtime (estimate = −0.16; $P = 0.002$; adjusted $R^2 = 0.97$; Fig. 7a). In contrast, when we consider the smallest dataset, k slightly increases runtime (estimate = 0.25; $P = 0.005$; adjusted $R^2 = 0.93$). As such, the effect of the number of partitions on DO runtime varies across dataset sizes, but in general partitioning reduces runtime in large datasets but not small datasets (Appendix S1: Fig. S2). Regarding tree costs, the range was 263 185–264 969 steps in the largest dataset. As expected, the relationship between tree cost and k is significantly positive (estimate = 1.78×10^{-3}; $P = 0.02$; adjusted $R^2 = 0.84$; Fig. 7b and Appendix S1: Fig. S3).

Empirical examples

We tested *prepDyn* using four empirical datasets, including Sanger, phylogenomic and hDNA sequences. We employed the following datasets: Neotropical torrent frogs (Hylodidae: 75 leaves, ca. 4291 bp comprising four Sanger fragments, Sá et al., 2022; preprocessing runtime = 0.73–1.16 CPU sec.), Amniota (16 leaves, ca. 187 026 bp comprising 248 protein-coding genes, Chiari et al., 2012; preprocessing runtime = 8.26–10.97 CPU sec.), Neotropical treefrogs (*Dendropsophus*: 115 leaves, ca. 6304 bp comprising 8 fragments of DNA, including Sanger and hDNA sequences, Nakamura et al., 2025; preprocessing runtime = 1.95–2.41 CPU sec.), and percamorph fishes (Percamorpha: 14 leaves, ca. 99 948 bp comprising 100 anchored hybrid enrichment [AHE] fragments, Dornburg et al., 2017; preprocessing runtime = 5.71–6.55

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

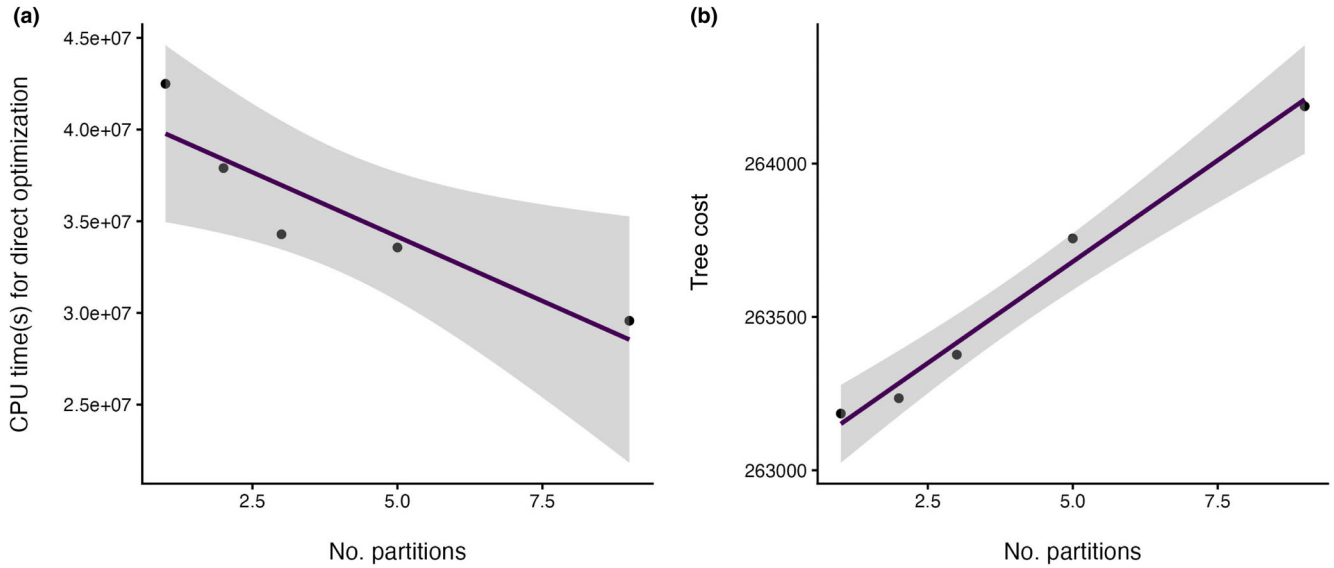


Fig. 7. Relationship between DO performance (runtime and tree cost) and number of partitions in the largest simulated dataset (80 leaves and 10 000 nucleotides). (a) CPU time using DO. (b) Tree costs using DO. See additional results in Appendix S1: Fig. S2 and Tables S1 and S2.

CPU sec). For each dataset, we employed six strategies: (1) no preprocessing, (2) preprocessing without partitioning, (3) preprocessing with maximum partitioning, (4) preprocessing with balanced partitioning, (5) preprocessing with conservative partitioning and (6) preprocessing with equal-length partitioning. Successive partitioning is applicable and was employed for Strategies 4–6.

We read all input data as unaligned sequences and ran a total of 252 equal-weighted parsimony analyses using POY v. 5.1.1 (Wheeler et al., 2015), PhyG v. 1.3 (Wheeler et al., 2024), and TNT v. 1.6 (Goloboff and Morales, 2023; Appendix S2). Initially, we conducted two timed searches in POY using the command `search` with a maximum runtime of 15 360 CPU hours (CPUh). Next, we inputted the POY trees as initial trees in PhyG to perform 480-CPUh searches using Thompson sampling with an exponential update function and memory factor of 2 (Wheeler, 2024). Finally, we submitted the implied alignments (Wheeler, 2003b; Washburn and Wheeler, 2020) with gaps as a fifth state to TNT using the command `xmult = level 10 chklevel 5 consense 5`. This combined POY + PhyG + TNT strategy enables analyses to leverage the strengths of all three programs; for example, POY enables parallel execution across multi-node, multi-core clusters (parallel execution is currently confined to single, multi-core machines in PhyG; Wheeler et al., 2024), whereas PhyG has efficient algorithms such as Thompson sampling (unavailable in POY; Wheeler, 2024), and TNT is extremely efficient at finding optimal solutions for a given alignment. In addition to comparing tree lengths, we computed normalized Robinson-Foulds (RF) distances between pairs of strict consensus trees using the function `RobinsonFoulds` from the package TreeDist v. 2.11.1 (Smith, 2020, 2022). Based on the minimum branch lengths mapped onto the strict consensus trees, we computed the proportion of different branch lengths between pairs of trees from Strategies 1–6 using the command `mapBranchLength` from the R package RNODE v. 0.2.0 (available at https://github.com/dnakamuraz/RNODE).

As expected, our results revealed that tree costs of analyses without preprocessing (Strategy 1) are higher than analyses with preprocessing (Strategies 2–6; Table 1). Contrary to simulation results, the tree costs of analyses with preprocessing but no partitioning (Strategy 2) did not return minimum-cost solutions. This is expected in large empirical datasets analysed with time-constrained searches.

The best partitioning strategy varied among empirical datasets. Specifically, the best tree costs were obtained using Strategies 2 (maximum partitioning; Percamorpha), 5 (conservative partitioning; Hylodidae) and 6 (equal-length partitioning; Amniota and *Dendropsophus*). Furthermore, successive partitioning played an important role in reducing tree costs in Strategies 4–6. For instance, in the Amniota dataset, we obtained the lowest tree cost using Strategy 6 after four rounds (194 372 steps in Round 4 vs 194 870 steps in Round 5; Appendix S2), whereas the worst performance among analyses with preprocessing was Strategy 2 (195 732 steps). In most cases, TNT analysis of the implied alignments did not improve tree costs relative to PhyG.

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 1
Tree costs from timed searches using empirical examples. Initial trees from POY analyses were refined in PhyG using Thompson sampling and in TNT using implied alignments (IA). Strategies: (1) without preprocessing, (2) with preprocessing, without partitioning, (3) with preprocessing, maximum partitioning, (4) with preprocessing, balanced partitioning, (5) with preprocessing, conservative partitioning and (6) with preprocessing, equal-length partitioning. The best round of successive partitioning is only applicable to Strategies 4–6. The best tree cost for each dataset is marked in bold. See expanded results in Appendix S2

Dataset	Preprocessing strategy	Best round	POY	PhyG	TNT (IA)
Sá et al. (2022)	1	NA	10 664	10 058	10 058
	2	NA	4748	4747	4747
	3	NA	4778	4773	4773
	4	1	4790	4782	4781
	5	**1**	**4748**	**4746**	**4746**
	6	1	4776	4772	4772
Chiari et al. (2012)	1	NA	353 482	350 651	350 651
	2	NA	196 269	195 732	195 732
	3	NA	195 153	194 638	194 638
	4	4	195 140	194 642	194 642
	5	2	196 410	195 648	195 648
	6	**4**	**195 035**	**194 372**	**194 372**
Dornburg et al. (2017)	1	NA	148 035	147 505	147 505
	2	NA	70 620	70 515	70 515
	3	**NA**	**70 561**	**70 443**	**70 443**
	4	1	70 629	70 495	70 495
	5	7	70 622	70 466	70 466
	6	4	70 633	70 480	70 480
Nakamura et al. (2025)	1	NA	19 773	19 535	19 534
	2	NA	13 688	13 474	13 474
	3	NA	13 720	13 468	13 467
	4	1	13 702	13 473	13 473
	5	2	13 689	13 466	13 466
	6	**5**	**13 520**	**13 461**	**13 461**

Normalized RF values were higher between Strategy 1 and other strategies, as expected due to the lack of identification of missing data in Strategy 1. Normalized RF values were zero (i.e. trees were identical) between most pairs of trees in Strategies 2–6 using phylogenomic datasets (Amniota and Percamorpha) but not the Sanger datasets (Hylodidae and *Dendropsophus*; Fig. 8). In contrast, a higher proportion of branch lengths differed among Strategies 2–6 using the phylogenomic datasets (77–100%) than the Sanger datasets (18–78%).

Discussion

In this study, we developed a method to facilitate data preprocessing for dynamic homology analysis. Collecting data from GenBank, trimming trivial data (e.g. flanking invariants and orphan nucleotides), identifying missing data, and partitioning can be performed using *prepDyn*. This method enables large numbers of loci in phylogenomics and hDNA sequences with a high proportion of missing data in museomics to be preprocessed automatically, which would otherwise be laborious and prone to human error if performed manually. Particularly, in phylogenomic data sequenced via target capture (e.g. ultraconserved elements and anchored hybrid enrichment; Faircloth et al., 2012; Lemmon et al., 2012), missing data are usually present in flanking positions of gene alignments (Carter et al., 2023). Previous attempts to automatically preprocess alignments have used tools like G-blocks (Castresana, 2000) or trimAl (Capella-Gutiérrez et al., 2009); however, those programs focus on trimming and lack the options for partitioning for dynamic homology, as implemented in *prepDyn*. They also lack options for aligning sequences containing non-standard symbols (e.g. question marks and hash signs, as implemented in *UP2AP*) and adding new sequences to profile alignments in POY/PhyG format (as implemented in *addSeq*).

Data preprocessing for dynamic homology has been a source of confusion for some users of POY/PhyG. For instance, De Laet (2010: 453) reported: "The problem [. . .] is that POY sometimes treats absence as missing data. Information input as an observed absence of a sequence—as a zero-length string—is interpreted as missing data in the cost calculations". However, zero-length partitions interpreted as missing data are not a flaw—it is precisely how POY (and now PhyG) is intended to work. These programs explain the observed variation among sequences by invoking evolutionary events; if part of the variation is known to be due to technical artefacts, then the user must provide that information. Missing data of a whole locus for a set of terminals is informed by the absence of this terminal when each locus is input as an individual file, thus treating missing data as zero-length partitions. In contrast, as clarified in *Preliminaries*, missing data within a locus can be coded during preprocessing using question marks or partitions. The former is used in both static and dynamic homology, whereas the latter is exclusive to dynamic homology. There are pros and cons to each method: coding missing data with question marks does not constrain homology testing, whereas partitioning does; partitioning is heuristically useful in reducing computational burden, whereas using only question marks can be computationally intensive.

Theoretically, within-gene partitioning should be minimized to maximize explanatory power, since partitioning imposes constraints on homology testing. However, our results corroborated Giribet's (2001) finding that partitioning can speed up empirical analyses, thereby enabling greater exploration of the tree-alignment space in a given amount of time. This is particularly useful for large datasets, including partial (e.g. AHE and UCE) or whole genomes. Given the practical limitations of time and computational resources, multiple timed searches using successive partitioning and/or different partitioning strategies should

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

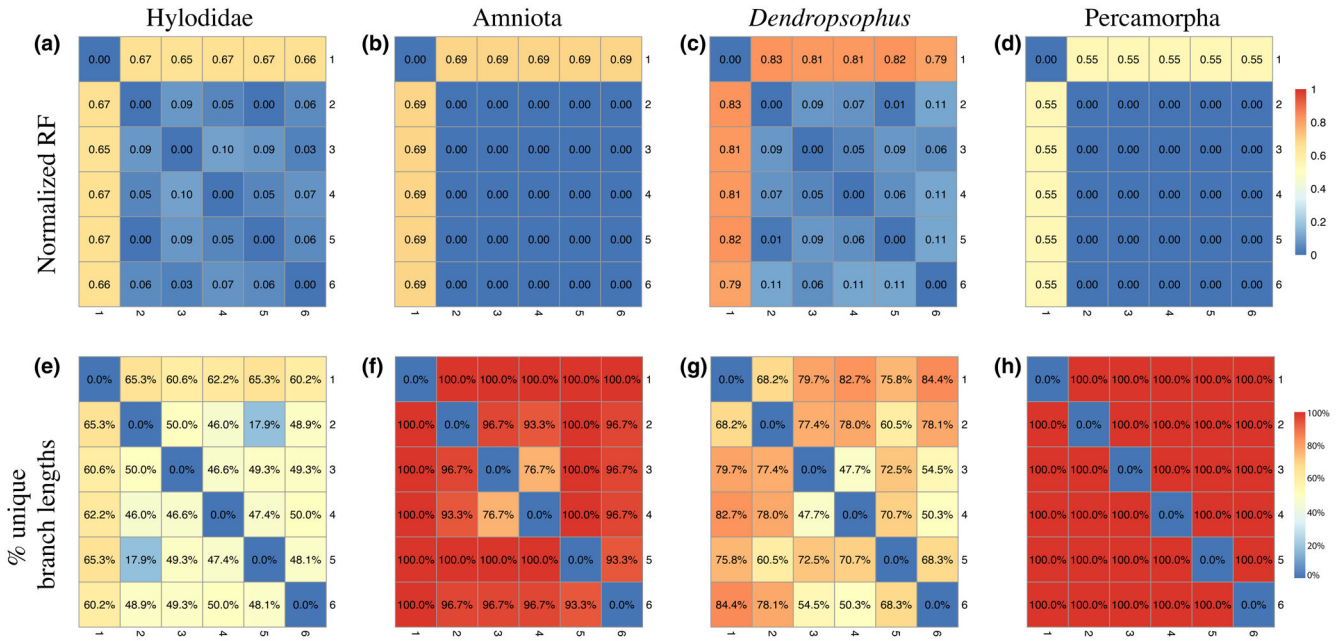


Fig. 8. Results of analysis of empirical datasets. Heatmaps showing (a–d) normalized Robinson-Foulds distances (zero values indicate that all clades are shared between two trees) and (e–h) proportions of unique branch-lengths (zero values indicate that all matching branches share the same lengths between two trees) between pairs of strategies in the (a, e) Hylodidae (Sá et al., 2022), (b, f) Amniota (Chiari et al., 2012), (c, g) *Dendropsophus* (Nakamura et al., 2025), and (d, h) Percamorpha (Dornburg et al., 2017) datasets. Strategies: (1) without preprocessing, (2) with preprocessing, without partitioning, (3) with preprocessing, maximum partitioning, (4) with preprocessing, balanced partitioning, (5) with preprocessing, conservative partitioning, and (6) with preprocessing, equal-length partitioning. See additional results in Appendix S2.

be conducted and their performance compared via tree costs.

In our empirical examples, conservative, equal-length and maximum partitioning achieved better tree costs in different datasets. The conservative strategy only inserts hash signs in the most conserved regions, thus minimizing constraints on homology tests. In contrast, equal-length partitioning changes the position of hash-sign columns in each round, which likely diversifies the exploration of the tree-alignment space. Finally, maximum partitioning inserts hash signs adjacent to all contiguous strings of missing data, thus reducing the computational burden in large datasets. Notably, although the two phylogenomic datasets were topologically identical across all partitioning strategies, most (Amniota) or all (Percamorpha) branch lengths differed across strategies, revealing the widespread impact of partition strategy on homology, which can affect downstream analyses that require branch length information (e.g. diversification rate estimation, phylogenetic regression; Revell and Harmon, 2022), even when the topology remains constant.

Although not evaluated in this study, other advantages of partitioning in dynamic homology analysis include the ability to reweight partitions in a modified ratcheting procedure (e.g. Wheeler et al., 2006), calculation of resampling support from unaligned sequences (although they can be computed from implied alignments), and softwired network search (Wheeler, 2015; Wheeler and Washburn, 2023; Wheeler et al., 2024). Methods incorporating higher-level genomic events (e.g. inversions and recombinations; Wheeler, 2007; Varón et al., 2010) have yet to be implemented in PhyG, but the procedures implemented in *prepDyn* are expected to improve costs in those cases as well. Likewise, in principle successive partitioning could be generalized to maximum likelihood under dynamic homology as long as the same data, model and form of likelihood (reviewed in Wheeler, 2006, 2012) are employed across partitioning rounds. However, alternate partition scenarios could impact model selection and estimation of parameters (e.g. branch lengths).

Conclusion

Three main points should be considered when using *prepDyn*. First, *prepDyn* automates DNA-sequence preprocessing by primarily coding missing data with question marks (instead of IUPAC Ns and dashes). If partitioning is performed, missing data can also be coded with zero-length partitions. Beyond POY/PhyG, if partitioning is skipped, *prepDyn* can be generalized to preprocess DNA sequences for other phylogenetic programs that treat gaps as a fifth character-state. Second, in addition to the number of leaves and nucleotides, the number of partitions should also be considered as a predictor of

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

time complexity when strategies of phylogenetic analyses are designed. In empirical datasets, multiple timed searches are necessary, and successive partitioning can improve tree costs. To accomplish this task, *prepDyn* facilitates the partitioning of dynamic characters using different methods (balanced, conserved, equal-length and maximum partitioning), which are comparable via tree costs. Finally, *prepDyn* automates steps previously performed manually, thus reducing subjectivity and human error and increasing replicability.

Acknowledgements

Financial support was provided by the Brazilian Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP grants 2018/15425-0 to TG, 2022/02789-0 and 2024/03494-9 to DYMN) and Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq 314480/2021-8 and 306625/2026-1 to TG). The American Museum of Natural History provided assistance to DYMN during his BEPE internship abroad. The Article Processing Charge for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614). The Article Processing Charge for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614).

Conflict of interest

None declared.

Data availability statement

Software and scripts to simulate data are available at https://github.com/dnakamuraz/PrepDyn. Manual installation of prepDyn requires Python v. 3.10.9 and MAFFT v. 7.5.2; other dependencies (including Bio v. 1.73, matplotlib v. 3.7.0, numpy v.1.23.5 and termcolor v. 3.2.0) should be installed automatically. Installation instructions via Conda and Docker are available at the GitHub repository. Tutorials are available at https://github.com/dnakamuraz/prepDyn/wiki. Empirical data are available in their original sources. RNODE v. 0.2.0 is available at https://github.com/dnakamuraz/RNODE.

References

Capella-Gutiérrez, S., Silla-Martínez, J.M. and Gabaldón, T., 2009. trimAl: a tool for automated alignment trimming in large-scale phylogenetic analyses. Bioinformatics 25, 1972–1973.

Carter, J.K., Kimball, R.T., Funk, E.R., Kane, N.C., Schield, D.R., Spellman, G.M. and Safran, R.J., 2023. Estimating phylogenies from genomes: a beginners review of commonly used genomic data in vertebrate phylogenomics. J. Hered. 114, 1–13.

Castresana, J., 2000. Selection of conserved blocks from multiple alignments for their use in phylogenetic analysis. Mol. Biol. Evol. 17, 540–552.

Chiari, Y., Cahais, V., Galtier, N. and Delsuc, F., 2012. Phylogenomic analyses support the position of turtles as the sister group of birds and crocodiles (Archosauria). BMC Biol. 10, 65.

Cock, P.J., Antao, T., Chang, J.T., Chapman, B.A., Cox, C.J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B. and de Hoon, M.J., 2009. Biopython: freely available Python tools for computational molecular biology and bioinformatics. Bioinformatics 25, 1422–1423.

De Laet, J., 2010. A problem in POY tree searches (and its workaround) when some sequences are observed to be absent in some terminals. Cladistics 26, 453–455.

Dornburg, A., Townsend, J.P., Brooks, W., Spriggs, E., Eytan, R.I., Moore, J.A., Wainwright, P.C., Lemmon, A., Lemmon, E.M. and Near, T.J., 2017. New insights on the sister lineage of percomorph fishes with an anchored hybrid enrichment dataset. Mol. Phylogenet. Evol. 110, 27–38.

Edgecombe, G.D., Giribet, G. and Wheeler, W.C., 2002. Phylogeny of Henicopidae (Chilopoda: Lithobiomorpha): a combined analysis of morphology and five molecular loci. Syst. Entomol. 27, 31–64.

Faircloth, B.C., McCormack, J.E., Crawford, N.G., Harvey, M.G., Brumfield, R.T. and Glenn, T.C., 2012. Ultraconserved elements anchor thousands of genetic markers spanning multiple evolutionary timescales. Syst. Biol. 61, 717–726.

Foulds, L.R. and Graham, R.L., 1982. The Steiner problem in phylogeny is NP-complete. Adv. Appl. Math. 3, 43–49.

Frost, D.R., Grant, T., Faivovich, J., Bain, R.H., Haas, A., De Sá, C.F.B.H.R.O., Channing, A., Wilkinson, M., Donnellan, S.C., Raxworthy, C.J., Campbell, J.A., Blotto, B.L., Moler, P., Drewes, R.C., Nussbaum, R.A., Lynch, J.D., Green, D.M. and Wheeler, A.W.C., 2006. The amphibian tree of life. Bull. Am. Mus. Nat. Hist. 297, 1–291.

Giribet, G., 2001. Exploring the behavior of POY, a program for direct optimization of molecular data. Cladistics 17, 60–70.

Giribet, G., 2002. Relationships among metazoan phyla as inferred from 18S rRNA sequence data: a methodological approach. In: DeSalle, R., Giribet, G. and Wheeler, W.C. (Eds.), Molecular Systematics and Evolution: Theory and Practice. Birkhauser Verlag, Basel, pp. 85–101.

Giribet, G., Distel, D.L., Polz, M., Sterrer, W. and Wheeler, W.C., 2000. Triploblastic relationships with emphasis on the acoelomates and the position of Gnathostomulida, Cycliophora, Plathelminthes, and Chaetognatha: a combined approach of 18S rDNA sequences and morphology. Syst. Biol. 49, 539–562.

Goloboff, P.A. and Morales, M.E., 2023. TNT version 1.6, with a graphical interface for MacOS and Linux, including new routines in parallel. Cladistics 39, 144–153.

Grant, T., Frost, D.R., Caldwell, J.P., Gagliardo, R., Haddad, C.F.B., Kok, P.J.R., Means, D.B., Noonan, B.P., Schargel, W.E. and Wheeler, W.C., 2006. Phylogenetic systematics of dart-poison frogs and their relatives (Amphibia: Athesphatanura: Dendrobatidae). Bull. Am. Mus. Nat. Hist. 299, 1–292.

Hein, J., 1989. A new method that simultaneously aligns and reconstructs ancestral sequences for any number of homologous sequences, when the phylogeny is given. Mol. Biol. Evol. 6, 649–668.

Hein, J., 1990. Unified approach to alignment and phylogenies. Methods Enzymol. 183, 626–645.

Hosner, P.A., Faircloth, B.C., Glenn, T.C., Braun, E.L. and Kimball, R.T., 2016. Avoiding missing data biases in phylogenomic inference: an empirical study in the landfowl (Aves: Galliformes). Mol. Biol. Evol. 33, 1110–1125.

Katoh, K. and Toh, H., 2008. Recent developments in the MAFFT multiple sequence alignment program. Brief. Bioinform. 9, 286–298.

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Kearse, M., Moir, R., Wilson, A., Stones-Havas, S., Cheung, M., Sturrock, S., Buxton, S., Cooper, A., Markowitz, S., Duran, C., Thierer, T., Ashton, B., Meintjes, P. and Drummond, A., 2012. Geneious Basic: an integrated and extendable desktop software platform for the organization and analysis of sequence data. Bioinformatics 28, 1647–1649.
Larkin, M.A., Blackshields, G., Brown, N.P., Chenna, R., McGettigan, P.A., Valentin, F., Wallace, I.M., Wilm, A., Lopez, R., Thompson, J.D., Gibson, T.J. and Higgins, D.G., 2007. Clustal W and Clustal X version 2.0. Bioinformatics 23, 2947–2948.
Lemmon, A.R., Emme, S.A. and Lemmon, E.M., 2012. Anchored hybrid enrichment for massively high-throughput phylogenomics. Syst. Biol. 61, 727–744.
Ly-Trong, N., Naser-Khdour, S., Lanfear, R. and Minh, B.Q., 2022. AliSim: a fast and versatile phylogenetic sequence simulator for the genomic era. Mol. Biol. Evol. 39, 1–9.
Nakamura, D.Y.M., Orrico, V.G.D., da Silva, E.M.G., Lyra, M.L. and Grant, T., 2025. Museomics reduces taxonomic inflation in the *Dendropsophus araguaya* complex (Hylinae: Dendropsophini) from the Cerrado. J. Vertebr. Biol. 74, 24112.1–24112.18.
Peloso, P.L., Frost, D.R., Richards, S.J., Rodrigues, M.T., Donnellan, S., Matsui, M., Raxworthy, C.J., Biju, S.D., Lemmon, E.M., Lemmon, A.R. and Wheeler, W.C., 2015. The impact of anchored phylogenomics and taxon sampling on phylogenetic inference in narrow-mouthed frogs (Anura, Microhylidae). Cladistics 32, 113–140.
Revell, L.J. and Harmon, L.J., 2022. Phylogenetic Comparative Methods in R. Princeton University Press, Princeton, NJ.
Sá, F.P., Condez, T.H., Lyra, M.L., Haddad, C.F.B. and Malagoli, L.R., 2022. Unveiling the diversity of Giant Neotropical Torrent frogs (Hylodidae): phylogenetic relationships, morphology, and the description of two new species. Syst. Biodivers. 20, 1–31.
Sankoff, D., 1975. Minimal mutation trees of sequences. SIAM J. Appl. Math. 28, 35–42.
Sankoff, D. and Rousseau, P., 1975. Locating the vertices of a Steiner tree in arbitrary space. Math. Program. 9, 240–246.
Sankoff, D., Cedergren, R.J. and Lapalme, G., 1976. Frequency of insertion-deletion, transversion, and transition in the evolution of 5S ribosomal RNA. J. Mol. Evol. 7, 133–149.
Smith, M.R., 2020. Information theoretic generalized Robinson–Foulds metrics for comparing phylogenetic trees. Bioinformatics 36, 5007–5013.
Smith, M.R., 2022. Robust analysis of phylogenetic tree space. Syst. Biol. 71, 1255–1270.
Tresoldi, T., 2021. Ngesh: a Python library for synthetic phylogenetic data. J. Open Source Softw. 6, 3173.
Varón, A. and Wheeler, W.C., 2012. The tree-alignment problem. BMC Bioinformatics 13, 293.
Varón, A. and Wheeler, W.C., 2013. Local search for the generalized tree alignment problem. BMC Bioinformatics 14, 66.
Varón, A., Vinh, L.S. and Wheeler, W.C., 2010. POY version 4: phylogenetic analysis using dynamic homologies. Cladistics 26, 72–85.
Wang, L. and Jiang, T., 1994. On the complexity of multiple sequence alignment. J. Comput. Biol. 1, 337–348.
Washburn, A.J. and Wheeler, W.C., 2020. Efficient implied alignment. BMC Bioinformatics 21, 296.
Wheeler, W.C., 1996. Optimization alignment: the end of multiple sequence alignment in phylogenetics? Cladistics 12, 1–9.
Wheeler, W.C., 2001. Homology and the optimization of DNA sequence data. Cladistics 17, 3–11.
Wheeler, W.C., 2003a. Iterative pass optimization of sequence data. Cladistics 19, 254–260.
Wheeler, W.C., 2003b. Search-based optimization. Cladistics 19, 348–355.
Wheeler, W.C., 2003c. Implied alignment: a synapomorphy-based multiple-sequence alignment method and its use in cladogram search. Cladistics 19, 261–268.
Wheeler, W.C., 2005. Alignment, dynamic homology, and optimization. In: Albert, V. (Ed.), Parsimony, Phylogeny, and Genomics. Oxford University Press, Oxford, pp. 71–80.
Wheeler, W.C., 2006. Dynamic homology and the likelihood criterion. Cladistics 22, 157–170.
Wheeler, W.C., 2007. Chromosomal character optimization. Mol. Phylogenet. Evol. 44, 1130–1140.
Wheeler, W.C., 2012. Systematics: A Course of Lectures. Wiley-Blackwell, Oxford.
Wheeler, W.C., 2015. Phylogenetic network analysis as a parsimony optimization problem. BMC Bioinformatics 16, 1–9.
Wheeler, W.C., 2024. Multi-armed bandits, Thomson sampling and unsupervised machine learning in phylogenetic graph search. Cladistics 40, 430–437.
Wheeler, W.C., 2025. The limits of phylogenetic analysis: identifying analytical hallucinations. Cladistics 41, 513–522.
Wheeler, W.C. and Washburn, A.J., 2023. Parsimony optimization of phylogenetic networks. Cladistics 39, 456–474.
Wheeler, W.C., Aagesen, L., Arango, C.P., Faivoich, J., Grant, T., D'Haese, C., Janies, D., Smith, W.L., Varón, A. and Giribet, G., 2006. Dynamic Homology and Systematics: A Unified Approach. American Museum of Natural History, New York, NY.
Wheeler, W.C., Lucaroni, N., Hong, L., Crowley, L.M. and Varón, A., 2014. POY 5.1.1. American Museum of Natural History. http://research.amnh.org/scicomp/projects/poy.php.
Wheeler, W.C., Lucaroni, N., Hong, L., Crowley, L.M. and Varón, A., 2015. POY version 5: phylogenetic analysis using dynamic homologies under multiple optimality criteria. Cladistics 31, 18–196.
Wheeler, W.C., Coddington, J.A., Crowley, L.M., Dimitrov, D., Goloboff, P.A., Griswold, C.E., Hormiga, G., Prendini, L., Ramírez, M.J., Sierwald, P., Almeida-Silva, L., Alvarez-Padilla, F., Arnedo, M.A., Benavides Silva, L.R., Benjamin, S.P., Bond, J.E., Grismado, C.J., Hasan, E., Hedin, M., Izquierdo, M.A., Labarque, F.M., Ledford, J., Lopardo, L., Maddison, W.P., Miller, J.A., Piacentini, L.N., Platnick, N.I., Polotow, D., Silva-Dávila, D., Scharff, N., Szűts, T., Ubick, D., Vink, C.J., Wood, H.M. and Zhang, J., 2017. The spider tree of life: Phylogeny of Araneae based on target-gene analyses from an extensive taxon sampling. Cladistics 33, 574–616.
Wheeler, W.C., Washburn, A. and Crowley, L.M., 2024. Phylogenetic Graph (PhyG): a new phylogenetic graph search and optimization program. Cladistics 40, 97–105.

Supporting Information

Additional supporting information may be found online in the Supporting Information section at the end of the article.

Appendix S1. Simulation results for time complexity.

Table S1. Selection of regression models using all simulated data to test the relationship between DO runtime and number of leaves (?), total sequence length (?) and number of partitions (?). The predictors (?:?) and (?:?) indicate statistical interactions. The same considering tree cost as response. Selected models in bold. See also Fig. S2 and Table S2.

Table S2. Statistical results of testing relationship between DO runtime and number of leaves (?), total sequence length (?), and number of partitions (?). The same considering tree cost as response. Regressions considering all data (following model selection in Table S1; see also Fig. S2), largest dataset (worst case with 80 leaves and 10 000 bp; see Fig. 7), and smallest dataset (simplest case with 10 leaves and 100 bp).

Fig. S1. Relationship between CPU runtime of preprocessing (prepDyn) and (A) number of leaves (10, 20, 40 and 80) and (B) total sequence length (100, 1000

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

and 10 000), considering hash sign columns (0, 1, 2, 4 and 8) using simulations. Scale was log-transformed to facilitate visualization. See also Fig. 6.

Fig. S2. Contour plot (3-dimensional relationship on a 2-dimensional surface) considering all data showing that runtime of direct optimization tends to be higher when longer sequences are less partitioned, whereas runtime tends to be lower when longer sequences are more partitioned. The effect of partitioning is reduced for smaller sequences. See also Tables S1 and S2.

Fig. S3. Relationship between tree costs and number of partitions considering all data. See also Tables S1 and S2 and visualization for the largest dataset alone in Fig. 7B.

Appendix S2. Expanded results using empirical datasets.

10960031, 0, Downloaded from https://onlinelibrary.wiley.com/doi/10.1111/cla.70076 by Daniel Yudi Miyahara Nakamura - University Of Sao Paulo - Brazil , Wiley Online Library on [26/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License